

INFO MP 02 : Piles

B Fleurier

Lycée Dessaignes

21 septembre 2016

1 I) Les structures connues

2 II) Les piles

3 III) Applications

I) Les structures connues 1) Les tableaux

Un tableau est une structure de données constituée d'un ensemble d'éléments ordonnés accessibles par leur position ou indice.

Dans les langages anciens ou simples, les éléments du tableau doivent tous être de même type. Python ne se plie pas à cette contrainte, on parle de tableaux hétérogènes.

Les caractéristiques des tableaux sont simples mais importantes :

- ▷ Une longueur fixe, correspondant à des éléments contigus en espace mémoire.
- ▷ Un accès immédiat à chaque élément, comme pour une variable simple.

2) Les listes chaînées

Une liste chaînée est une structure de données constituée d'un ensemble d'éléments ordonnés dont l'accès se fait de manière séquentielle. Chaque élément de la liste permet d'accéder au suivant.

Chaque élément du tableau, en plus de sa valeur possède un pointeur vers l'adresse de l'élément suivant.

Les caractéristiques des listes chaînées sont simples mais importantes :

- ▷ Une longueur arbitraire, on peut ajouter ou enlever des éléments simplement en début et fin de liste.
- ▷ Un accès limité aux éléments centraux, il faut parcourir les éléments les précédents avec d'y accéder.

3) Les tableaux redimensionnables

On l'a vu, en Python les types précédents se confondent un peu (et c'est pas fini), on peut accéder facilement à un élément du tableau, mais on peut modifier sa longueur !

On parle de tableaux redimensionnables, alliant les forces des deux, sans leur faiblesses. L'intérêt est qu'on peut, par contrainte ou choix, se limiter dans les possibilités et les considérer comme un simple tableau ou liste chaînée.

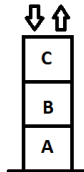
Concrètement comment fait Python ? Il "triche".

Il utilise effectivement des tableaux de taille fixe, de taille supérieur au nombre de valeurs à traiter dans notre tableaux. Quand on dépasse la taille allouée, il construit un nouveau tableau deux fois plus grand dans lequel il inclut l'ancien.

II) Les piles 1) Structure

On va s'intéresser à nouvelle structure de données : les piles. Le principe est basé sur le "dernier arrivé, premier sorti", on parle de LIFO (Last In, First Out). L'image traditionnelle est une pile d'assiette.

C est empilé sur B qui est empilé sur A. On peut soit retirer C de la pile (on dépile, ou pop), soit ajouter un élément D au sommet de la pile (on empile, ou push).



On va tout d'abord définir les opérations nécessaires à l'utilisation de piles :

- ▷ `creer_pile(N)` : crée une pile de capacité N , initialement vide.
- ▷ `depiler(p)` : dépile et renvoie le sommet de la pile p .
- ▷ `empiler(p,v)` : empile la valeur v sur la pile p .

On illustre ça :

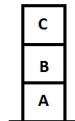
```
p = creer\_pile(10)
```

```
empiler(p,A)
```

```
empiler(p,B)
```

```
empiler(p,C)
```

```
depiler(p)
```



Les opérations empiler et depiler modifient le contenu de la pile.

D'autres opérations simples ne modifient pas la pile :

- ▷ `taille(p)` : renvoie le nombre d'éléments contenus dans p .
- ▷ `est_vide(p)` : indique si la pile p est vide.
- ▷ `sommet(p)` : renvoie le sommet de la pile sans la modifier.

Quelque soit la manière dont on réalise la structure de pile, ces opérations doivent se faire rapidement et simplement.

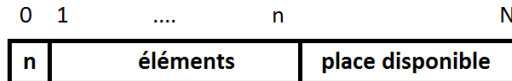
Une pile est (maintenant) une idée : peu importe si en machine on utilise des tableaux ou autre, si on considère que ce sont des piles, ce sont des piles.

Mais c'est justement les particularités des piles qui en feront un outil utile sur certains problèmes.

2) Piles bornées

La manière la plus simple de réaliser une pile consiste à utiliser un tableau de taille N avec N suffisamment grand pour contenir les éléments hypothétiques de la pile.

Pour empiler et dépile, il faut connaître la position du sommet de la pile dans le tableau. Pour cela le plus simple est de stocker le nombre d'éléments n de la pile dans la case 0 du tableau, puis les n éléments de la pile.



Création d'une pile On construit une pile de capacité N , constitué de `None`, et on rentre le nombre d'éléments (0) en case 0.

```
def creer_pile(N):  
    p = (N + 1) * [None]  
    p[0] = 0  
    return(p)
```

Dépiler un élément On repère le nombre d'éléments en case 0, on s'assure que la pile n'est pas vide, on passe le nombre d'éléments de n à $n-1$ et on retourne l'élément concerné.

```
def depiler(p):  
    n= p[0]  
    assert n > 0  
    p[0] = n -1  
    return(p[n])
```

$p[n]$ reste dans le tableau : osez, le nombre d'éléments est $n-1$.

Empiler un élément On doit d'abord tester si il reste de la place pour placer un élément dans le tableau, puis on stocke la nouvelle valeur et on indique le nouveau nombre d'éléments.

```
def empiler(p,v):  
    n= p[0]  
    assert n < len(p)-1  
    n = n+1  
    p[0] = n  
    p[n] = v
```

On peut définir les dernières opérations facilement :

```
def taille(p):  
    return(p[0])  
  
def est_vide(p):  
    return(taille(p)==0)  
  
def sommet(p):  
    assert taille(p) > 0  
    return(p[p[0]])
```

3) Piles non bornées

Le gros défaut de la structure de pile précédente est l'aspect bornée de la pile.

Et là, Python nous viens en aide, on va utiliser la taille variables des tableaux à foison. On utilise toutes les astuces Python, dont les commandes `pop` et `append`. On réécrit le tout avec les commandes qu'on a définies, mais en pratique on s'en passe largement.

```
def creer_pile(N):  
    return([])
```

```
def est_vide(p):  
    return(taille(p)==0)
```

```
def depiler(p):  
    assert len(p) > 0  
    return( p.pop())
```

```
def sommet(p):  
    assert len(p) > 0  
    return(p[-1])
```

```
def empiler(p,v):  
    p.append(v)
```

```
def taille(p):  
    return(len(p))
```

III) Applications 1) Mots bien parenthésés

Considérons une chaîne de caractères ne comportant que les symboles '(' et ')' (on fera avec le reste en TP).

On veut déterminer si un mot est bien parenthésé : c'est à dire le mot vide, soit la concaténation de deux mots bien parenthésés, soit un mot bien parenthésé mis entre parenthèses.

Par exemple : "", '()()' et '(()())' sont bien parenthésés, '(()', '())(' ne le sont pas.

On veut de plus que pour chaque parenthèse ouvrante, on renvoie le couple correspondant à la position de cette parenthèse et celle de sa parenthèse fermante correspondante.

On va le traiter avec des piles, en utilisant la version bornée. Pour ce faire avec un mot s , on crée une pile de longueur $\text{len}(s)$, puis on parcourt la chaîne à l'aide d'un `for`. Si c'est une parenthèse ouvrante, on empile, sinon (comme il n'y que '(' et ')') c'est une parenthèse fermante, on vérifie et on dépile, et si tout est bon, et on affiche les positions.

```
def parenthese(s):  
    p = creer_pile(len(s))  
    for i in range(len(s)):  
        if s[i] == '(' :  
            empiler(p,i)  
        else:  
            if est_vide(p):  
                return(False)  
            j = depiler(p)  
            print((j,i))  
    return(est_vide(p))
```

2) Notation polonaise inverse

La notation polonaise inverse (NPI) est une notation où les opérateurs arithmétiques sont placés après les opérandes : $2 + 3$ s'écrit $23+$, et $2 + 3 * 4$ s'écrit $234*+$. L'avantage est que les parenthèses sont inutiles : $(2 + 3) * 4$ s'écrit $23 + 4*$.

On représente une expression NPI par un tableau contenant les entiers et caractères (on se restreint à $+$ et $*$), par exemple $[1, 2, '+', 3, '*', '']$ pour $12 + 3*$.

Evaluer une NPIP nécessite l'utilisation d'une pile : on empile les nombres rencontrés, et lorsque l'on rencontre un opérateur on dépile, on calcule et on empile le résultat.

On se passe des commandes, en écrivant en Python directement.

```
def eval_npi(exp):  
    p = []  
    for c in exp :  
        if c == '+' or c == '*':  
            y = p.pop()  
            x = p.pop()  
            if c == '+':  
                z = x + y  
            else:  
                z = x * y  
            p.append(z)  
        else:  
            p.append(c)  
    v = p.pop()  
    assert(p == [])  
    return v
```